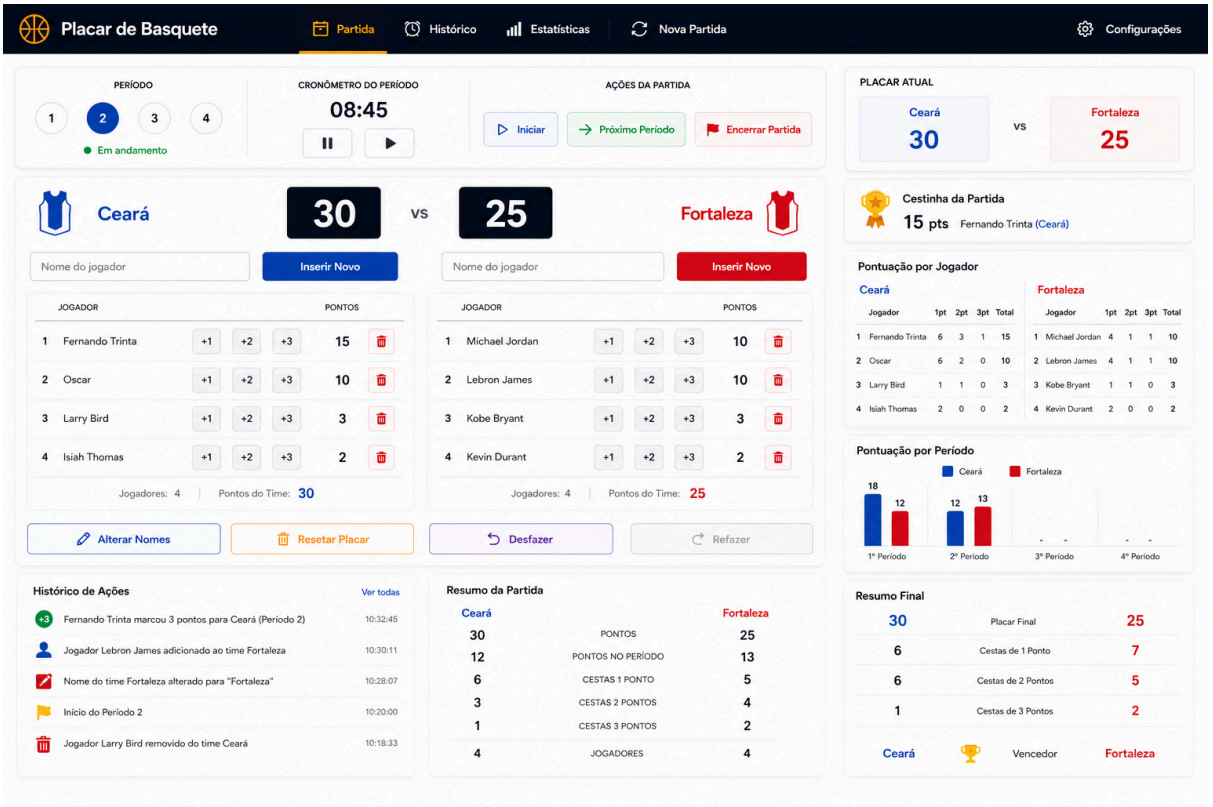
Especificação do Trabalho Prático React
Desenvolvimento de Software para Web

Desenvolva uma aplicação Web utilizando React com Next.js para controlar o placar de uma partida de basquete. A aplicação deve permitir o gerenciamento das equipes, dos jogadores, da pontuação individual e coletiva, do tempo/período da partida, do histórico de ações e das estatísticas do jogo.O objetivo principal do trabalho é praticar os conceitos de componentização, passagem de dados entre componentes, hooks, Context API e useReducer.

Uma **sugestão** de como o placar deve ser apresentado, é ilustrada na figura abaixo:



A aplicação deverá permitir o controle de uma partida entre duas equipes. Cada equipe deverá possuir um nome, que poderá ser alterado pelo usuário, e uma lista de jogadores participantes da partida. O usuário deverá poder cadastrar novos jogadores em cada equipe, informando o nome do jogador e associando-o ao time correspondente. A aplicação deverá validar esse cadastro, impedindo a inserção de jogadores com nome vazio ou repetido dentro da mesma equipe. Jogadores também podem ser removidos de suas equipes ou ter seu nome alterado.

Durante a partida, cada jogador poderá receber pontuações de 1, 2 ou 3 pontos, de acordo com os botões associados ao seu nome. Sempre que um jogador pontuar, a aplicação deverá atualizar tanto

sua pontuação individual quanto o placar total da equipe. Os jogadores de cada equipe deverão ser exibidos em ordem decrescente de pontuação, de modo que os maiores pontuadores apareçam no topo da lista.

Além do controle básico de pontos, a aplicação deverá representar a evolução da partida em quatro períodos de 10 minutos. O usuário deverá poder iniciar, pausar, avançar o período e encerrar a partida. Cada pontuação registrada deverá estar associada ao período em que ocorreu. Após o encerramento da partida, a aplicação não deverá permitir novas pontuações.

A aplicação também deverá manter um histórico das ações realizadas pelo usuário, incluindo inserção de jogadores, alteração dos nomes das equipes, registro de pontos, avanço de período e reinício da partida. Esse histórico deverá ser apresentado na interface. O sistema deverá permitir desfazer a última ação realizada, restaurando a aplicação ao estado imediatamente anterior. Como funcionalidade adicional, poderá ser implementada a opção de refazer uma ação desfeita.

Ao longo da partida, a aplicação deverá apresentar estatísticas gerais do jogo, tais como o placar total de cada equipe, o cestinha da partida, a pontuação individual dos jogadores, a quantidade de cestas de 1, 2 e 3 pontos por jogador, o total de pontos por período e, ao final, a equipe vencedora. Este painel pode estar vísivel ou não, de acordo com o desejo do usuário de usar ou não a tela toda. Da mesma forma, o painel de histórico de ações pode ou não ser omitido.

A aplicação deverá salvar automaticamente o estado da partida no navegador utilizando `localStorage`. Dessa forma, caso a página seja recarregada, a partida em andamento deverá ser restaurada. Também deverá existir uma opção para iniciar uma nova partida, limpando os dados salvos e retornando a aplicação ao seu estado inicial.

Do ponto de vista técnico, a solução deverá utilizar obrigatoriamente React com Next.js, com uma organização adequada em componentes. O estado principal da partida deverá ser gerenciado com `useReducer`, enquanto a Context API deverá ser utilizada para compartilhar esse estado e as ações da aplicação entre os diferentes componentes. O hook `useEffect` deverá ser utilizado para lidar com a persistência dos dados no `localStorage`. O uso de `useState` é permitido para estados locais simples, como campos de formulário ou controle de interface.

Não é permitido utilizar bibliotecas externas de gerenciamento de estado, como Redux, Zustand ou MobX. Também não devem ser utilizadas bibliotecas JavaScript externas para implementar a lógica da aplicação. O aluno poderá utilizar CSS próprio ou Bootstrap para auxiliar na construção da interface.

A avaliação levará em consideração o funcionamento correto da aplicação, a organização do código, a divisão adequada em componentes, o uso coerente de `useReducer`, Context API e hooks, a implementação das validações, a persistência dos dados, a clareza da interface e a qualidade geral da solução apresentada. O layout da aplicação é de responsabilidade do aluno, podendo seguir ou não o modelo apresentado em sala, desde que todas as funcionalidades descritas sejam implementadas.